Assignment 2: MPI Adaptive Numerical Integration

Name: Eddie Ogendo

ID: 3736377

Date: March 2026

Github: [edd-ie/Asn2-CS4575]

---

1. Introduction

The objective of this assignment was to implement an **adaptive Simpson's Rule** using MPI to evaluate parallel performance through static and dynamic load balancing. I tested three distinct functions to analyze how varying computational workloads and irregular "spikes" in complexity affect parallel efficiency

2. Methodology

2.1 Hardware and Software Environment

- **Machine:** Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz
- **Environment:** 8-core configuration via WSL Ubuntu 24.04
- **Compilation:** Compiled using `mpicc` with `-O3` and `-fopenmp` optimization flags.

2.2 Implementation Modes

- **Mode 1 (Static):** The integration range [0, 1] is divided into P equal segments. Each process independently integrates its assigned segment, and results are aggregated via MPI_Reduce.
- **Mode 2 (Dynamic):** A Master-Worker architecture where Rank 0 manages a task stack. Workers (Ranks 1 to P-1) request work using TAG_WORK_REQUEST, ensuring they only receive tasks when idle.

---

3. Performance Results

The table below summarizes the core performance metrics. Speedup (S) is calculated as $S(P) = T_{serial} / T_{parallel}$

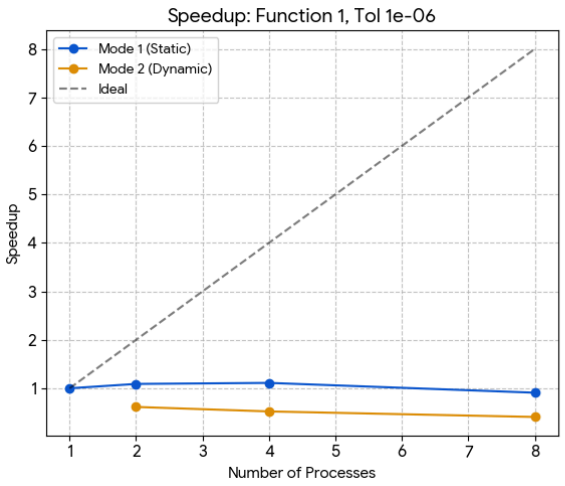
Function	Mode	Tolerance	Processes	AveTime(s)	T_serial	Speedup	Efficiency
1	1	0.000001	1	0.00023	0.00023	1	1
1	1	0.000001	2	0.000211	0.00023	1.090047	0.545023

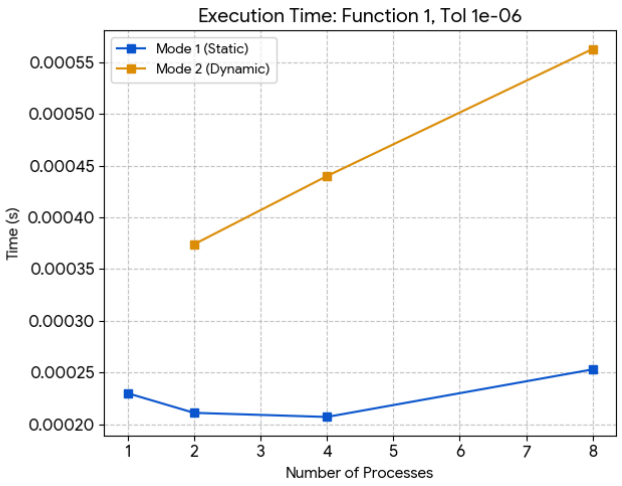
						393	6967
1	1	0.000001	4	0.000207	0.00023	1.111111111	0.2777777778
1	1	0.000001	8	0.000253	0.00023	0.9090909091	0.1136363636
1	1	0.00000001	1	0.000719	0.000719	1	1
1	1	0.00000001	2	0.000558	0.000719	1.288530466	0.644265233
1	1	0.00000001	4	0.000405	0.000719	1.775308642	0.4438271605
1	1	0.00000001	8	0.000418	0.000719	1.720095694	0.2150119617
1	2	0.000001	2	0.000374	0.00023	0.614973262	0.307486631
1	2	0.000001	4	0.00044	0.00023	0.5227272727	0.1306818182
1	2	0.000001	8	0.000563	0.00023	0.4085257549	0.05106571936
1	2	0.00000001	2	0.001028	0.000719	0.6994163424	0.3497081712
1	2	0.00000001	4	0.000768	0.000719	0.9361979167	0.2340494792
1	2	0.00000001	8	0.001055	0.000719	0.6815165877	0.08518957346
2	1	0.000001	1	0.000046	0.000046	1	1
2	1	0.000001	2	0.000084	0.000046	0.5476190476	0.2738095238
2	1	0.000001	4	0.000315	0.000046	0.146031746	0.03650793651
2	1	0.000001	8	0.00183	0.000046	0.02513661202	0.003142076503
2	1	0.00000001	1	0.000097	0.000097	1	1
2	1	0.00000001	2	0.000102	0.000097	0.9509803922	0.4754901961
2	1	0.00000001	4	0.000159	0.000097	0.6100628931	0.1525157233
2	1	0.00000001	8	0.000231	0.000097	0.4199134199	0.05248917749
2	2	0.000001	2	0.000156	0.000046	0.2948717949	0.1474358974
2	2	0.000001	4	0.000145	0.000046	0.317241	0.079310

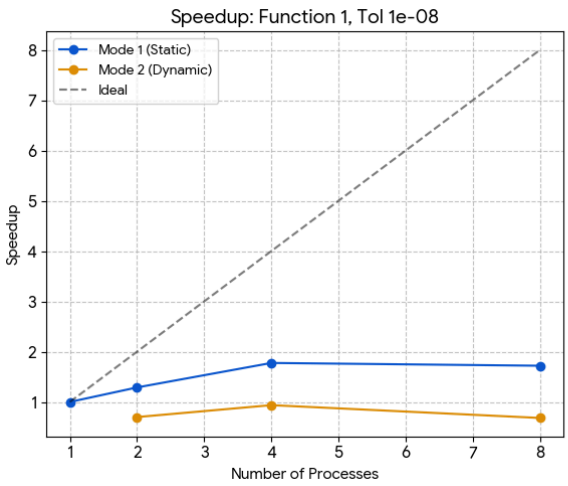
						3793	34483
2	2	0.000001	8	0.000326	0.000046	0.141104 2945	0.017638 03681
2	2	0.00000001	2	0.000202	0.000097	0.480198 0198	0.240099 0099
2	2	0.00000001	4	0.00028	0.000097	0.346428 5714	0.086607 14286
2	2	0.00000001	8	0.00065	0.000097	0.149230 7692	0.018653 84615

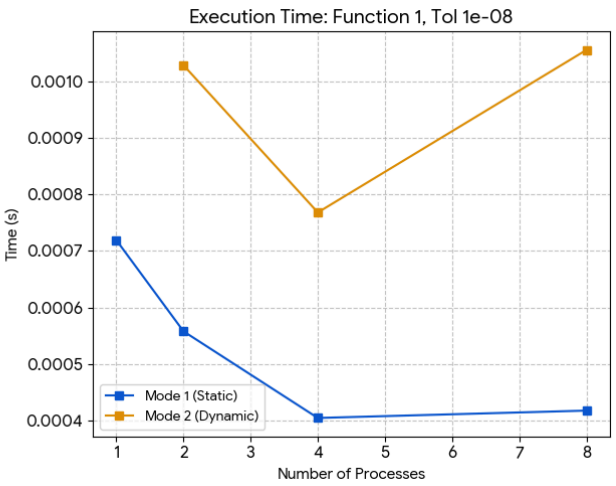
3.1 Speedup and Efficiency

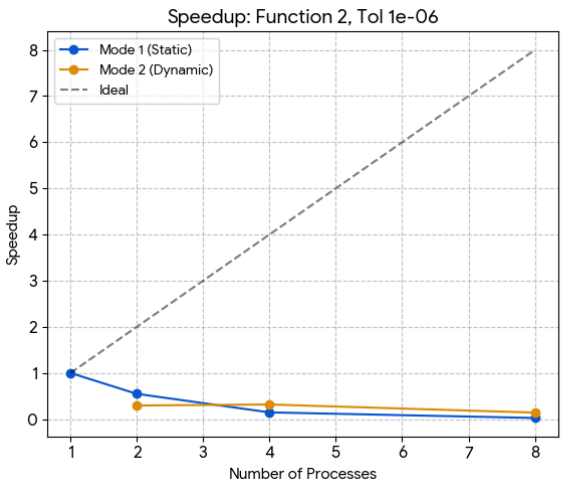
- **Speedup (S):**
  $S(P) = T_{serial} / T_{parallel}$
- **Efficiency (E):**
  $E(P) = S(P) / P$

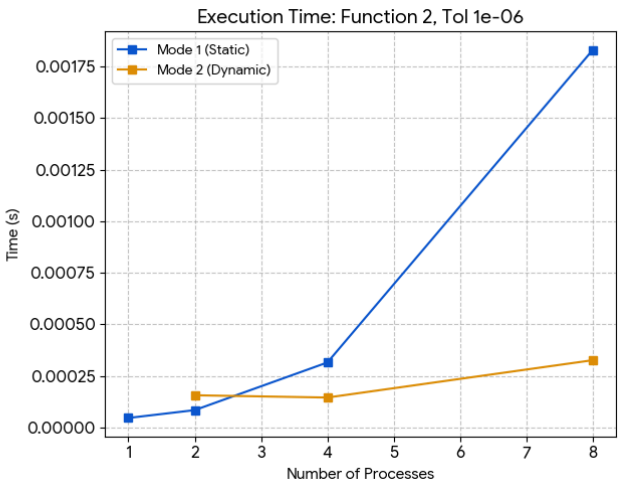


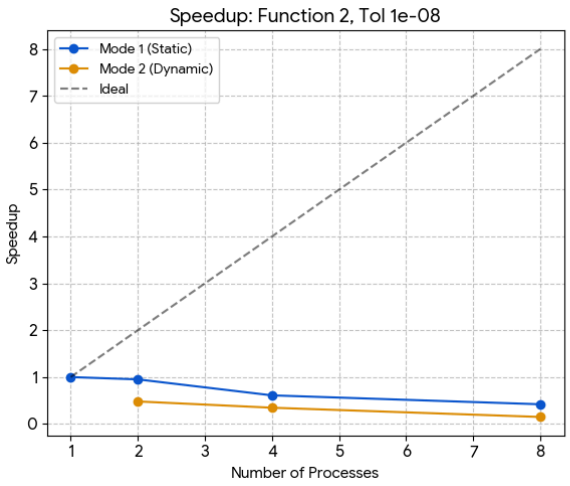


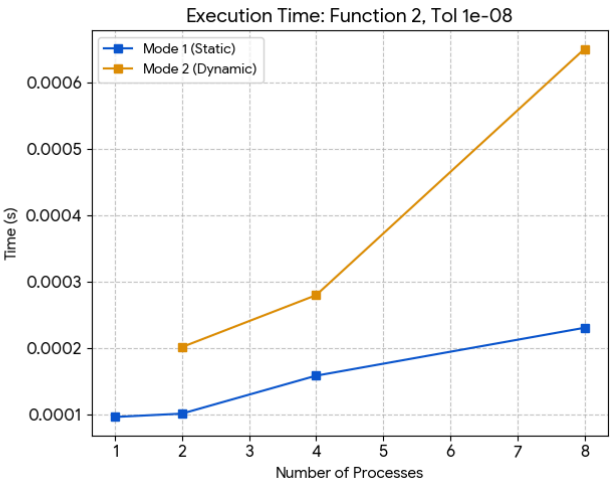












4. Discussion and Analysis

4.1 Function 1: Load Imbalance and Efficiency

For Function 1, the adaptive algorithm concentrates work around the "spike" at x=0.3.

- **Mode 1 (Static):** Showed marginal speedup up to P=4 (1.11x) but efficiency dropped to 28%. When P=8, the time actually increased because the segments became so small that the overhead of launching tasks and calling MPI_Reduce eclipsed the computational savings.
- **Mode 2 (Dynamic):** Performance was generally lower than Mode 1 for this function. This is because Rank 0 acts purely as a manager, meaning at P=2, only 50% of the resources are computing.

4.2 Function 2: Communication Dominance (Negative Scaling)

Function 2 is affected by **granularity**:

- The execution time for P=1 is extremely low.
- In Mode 1 at P=8, the execution time increases to 1.83 ms, representing a 40x slowdown (Speedup = 0.025). This occurs because the time spent by the OS to

synchronize 8 MPI processes is orders of magnitude larger than the time required to compute the integral.
- **Observation:** Dynamic load balancing (Mode 2) shows better performance than Static in some high-tolerance cases because it avoids "starvation" of processes, but it still suffers from the same overhead constraints.

---

5. Challenges and Solutions

- **MPI Slots:** Used `--oversubscribe` to bypass process limits in the local WSL environment.
- **Granularity:** Implemented "dummy work" to increase the computational load, allowing for the observation of meaningful scaling trends that would otherwise be masked by communication latency.
- **Termination:** The Master utilized a `TAG_STOP` signal to shut down workers once the task stack was exhausted and all workers reported as idle.

.

6. Conclusion

Static load balancing (Mode 1) is efficient for predictable, uniform workloads but fails when faced with irregular functions. Dynamic load balancing (Mode 2), despite higher communication costs and the loss of one computing rank, is essential for computationally expensive, irregular tasks like adaptive integration.